

トランスフォーム情報を表示

日本工学院専門学校
デザインカレッジ

```
import math
```

```
#シーン内の全オブジェクトについて
for object in bpy.context.scene.objects:
    print(object.type + " - " + object.name)
    #ローカルトランスフォーム行列から平行移動、回転、スケーリングを抽出
    #型は Vector, Quaternion, Vector
    trans, rot, scale = object.matrix_local.decompose()
    #回転を Quaternion から Euler (3軸での回転角) に変換
    rot = rot.to_euler()
    #ラジアンから度数法に変換
    rot.x = math.degrees(rot.x)
    rot.y = math.degrees(rot.y)
    rot.z = math.degrees(rot.z)
    次ページへつづく
```

python の標準数学モジュールを
使うため、
math をインポート。

下側のプログラムは
[13ページ](#) のfor文の拡張。

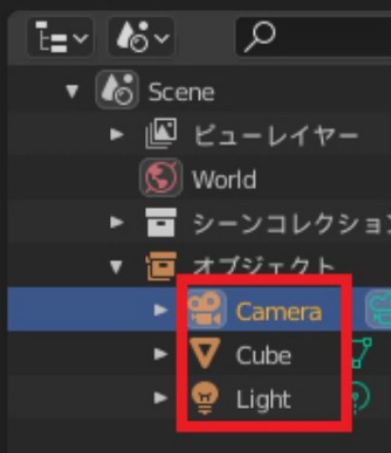
ベクトル、行列、クォータニオンは math ではなく、
Blender Python API 側で、
[mathutils](#) として実装されている。

ラジアンから度数法への変換は
math モジュールにある。

実行結果

日本工学院専門学校
デザインカレッジ

シーン情報をExportします
MESH - Cube
LIGHT - Light
CAMERA - Camera
シーン情報をExportしました



Blenderで新規作成した直後のシーンで
「シーン出力」を実行すると、

このような結果になる。

- 種類がMESH のオブジェクト「Cube」
- 種類がLIGHTのオブジェクト「Light」
- 種類がCAMERAのオブジェクト「Camera」

の3つがある。

オブジェクトを増やして実行すると
増やした分も表示される。

全オブジェクト走査

日本工学院専門学校
デザインカレッジ

```
def execute(self, context):  
    print("シーン情報をExportします")  
  
    #シーン内の全オブジェクトについて  
    for object in bpy.context.scene.objects:  
        print(object.type + " - " + object.name)  
  
    print("シーン情報をExportしました")  
    self.report({'INFO'}, "シーン情報をExportしました")  
  
    return {'FINISHED'}
```

MYADDON_OT_export_scene
クラスの execute()
を書き換えて、表示する情報を
増やしていく。

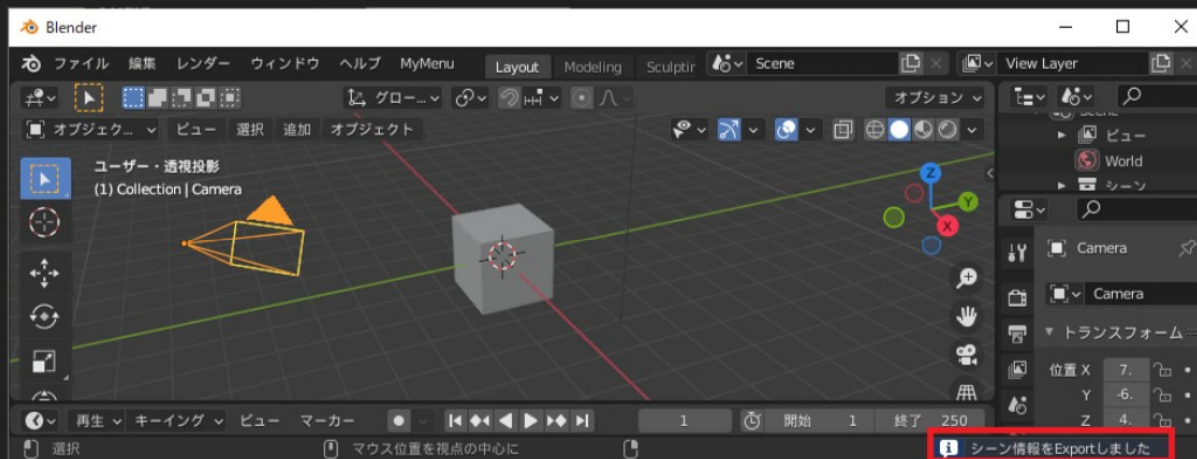
bpy.context.scene.objects は
コレクション（動的配列）
なので、for文でオブジェクトを
1個ずつ取り出しながら
処理を回すことができる。

今回はオブジェクトの種類と
名前の表示を追加した。

[リファレンス](#)

self.report()

日本工学院専門学校
デザインカレッジ



self.report() はBlenderのウィンドウ下部に出るメッセージを出力する機能。

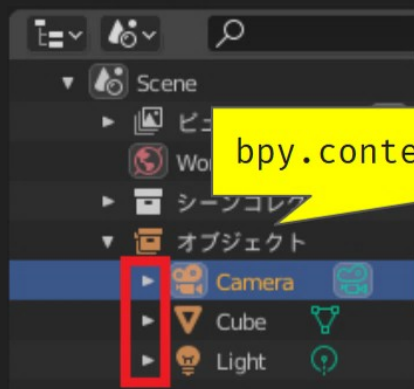
ツールを使う普通のユーザはコンソールウィンドウなど見ないので、ユーザに伝えたい情報を出力するのに重要。

実行結果

日本工学院専門学校
デザインカレッジ



```
問題 3 出力 デバッグコンソール ターミナル
module changed on disk: 'g:\Applications\B
register_class(...):
警告: 「topbar_mt_my_menu」に「_MT_」と、接続
レベルエディタが有効化されました。
<bpy_collection[3], SceneObjects>
シーン情報をExportしました
レベルエディタが無効化されました。
module changed on disk: 'g:\Applications\B
レベルエディタが有効化されました。
<bpy_collection[3], SceneObjects>
シーン情報をExportしました
```



bpy.context.scene.objects

アドオンを無効化→有効化し、
メニューから「シーン出力」を実行すると、
コンソールにprint() による情報が
表示される。

<bpy_collection[3], SceneObjects>

というのは、
`bpy.context.scene.objects` が
要素数3個の配列(std::vectorみたいなやつ)
であることを表している。

つまり、シーン内のオブジェクトのリスト。

オペレータクラス

日本工学院専門学校
デザインカレッジ

```
level_editor.py
#オペレータ シーン出力
class MYADDON_OT_export_scene(bpy.types.Operator):
    bl_idname = "myaddon.myaddon_ot_export_scene"
    bl_label = "シーン出力"
    bl_description = "シーン情報をExportします"

    def execute(self, context):

        print("シーン情報をExportします")

        print(bpy.context.scene.objects)

        print("シーン情報をExportしました")
        self.report({'INFO'}, "シーン情報をExportしました")

        return {'FINISHED'}
```

bpy.types.Operator を継承して
新たなオペレータのクラスを定義す
る。

bpy.context.scene は
Blenderで編集集中の
シーンのこと。
[リファレンス](#)

差し当たりシーンにある
オブジェクトリストを
print() で表示してみる。

前回反復した、オペレータをメ
ニューに登録する処理もお忘れな
く。

シーン情報の走査

日本工学院専門学校
デザインカレッジ

今回、Blenderのアドオンによってレベルエディタを作るということで、Blenderのシーンをそのままゲーム用のレベルデータとして出力する方針。

重要なこととして

- シーンにあるものを全て走査して情報収集
- 収集した情報を一覧としてファイルに書き出す

ということが挙げられる。

ファイル出力は後回しにして、まずはシーン走査し
コンソールに情報を表示してみたい。

走査：対象（画像、物体など）を点や線状に分解し、端から端まで順次なぞることで
情報を電気信号に変換（取り込み）または映像として再生・表示する技術

シーン情報の走査

レベルエディタの類義語

日本工学院専門学校
デザインカレッジ

レベルエディタと、シーンエディタ、ステージエディタ、マップエディタはほぼ同じ意味だと思ってよい。

大がかりな3Dゲームではエディタ上で建物や地形を並べていくとともに、

- イベントのトリガー
- ドアや鍵、テレポーターなどのギミックの配置
- 敵、モンスターの発生ポイント
- プレイヤーキャラの出現位置

など、ゲームの遊びに密接に関わる様々なものもツール上で配置して調整数値も設定するなど、文字通りレベルデザインをツール上で行う。

これらはエディタ単体では機能しないので、レベルエディタから出力したデータを使用するゲーム側のプログラムとの連携が重要である。

レベルエディタ

日本工学院専門学校
デザインカレッジ

- パズルゲームの1ステージ
- アクションゲームのストーリー1ステージ
- RPGのダンジョン1階層（ロードを挟まずに動く範囲）
- RPGの街1つ（ロードを挟まずに動く範囲）

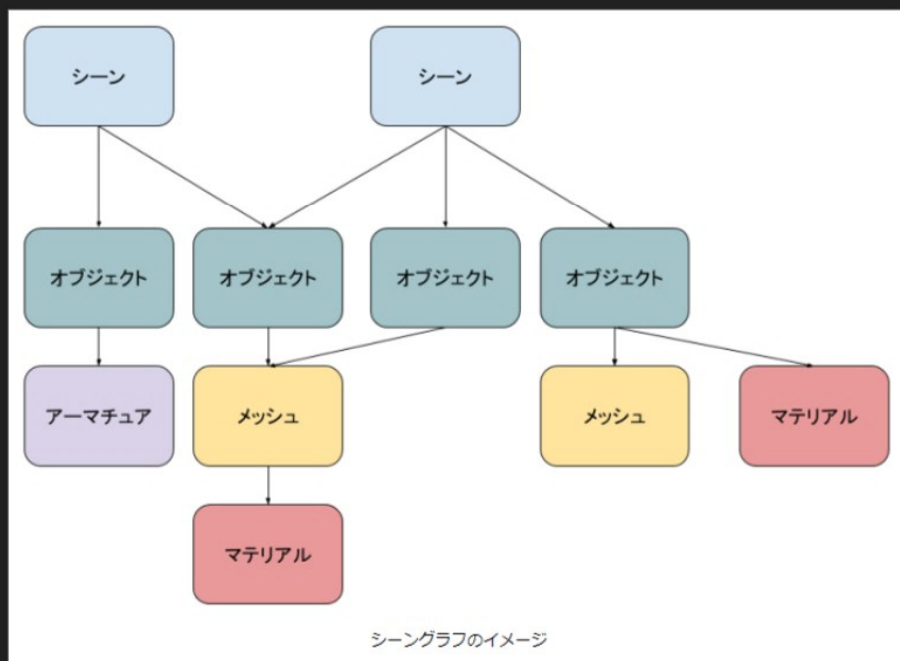
などのことを、レベル(Level)と呼ぶ。
（RPGの主人公のレベルと混同しないこと）

レベルデザインのレベルと同じ用語。
レベルデザインを行う為のツールがレベルエディタ。



シーングラフ

日本工学院専門学校
デザインカレッジ



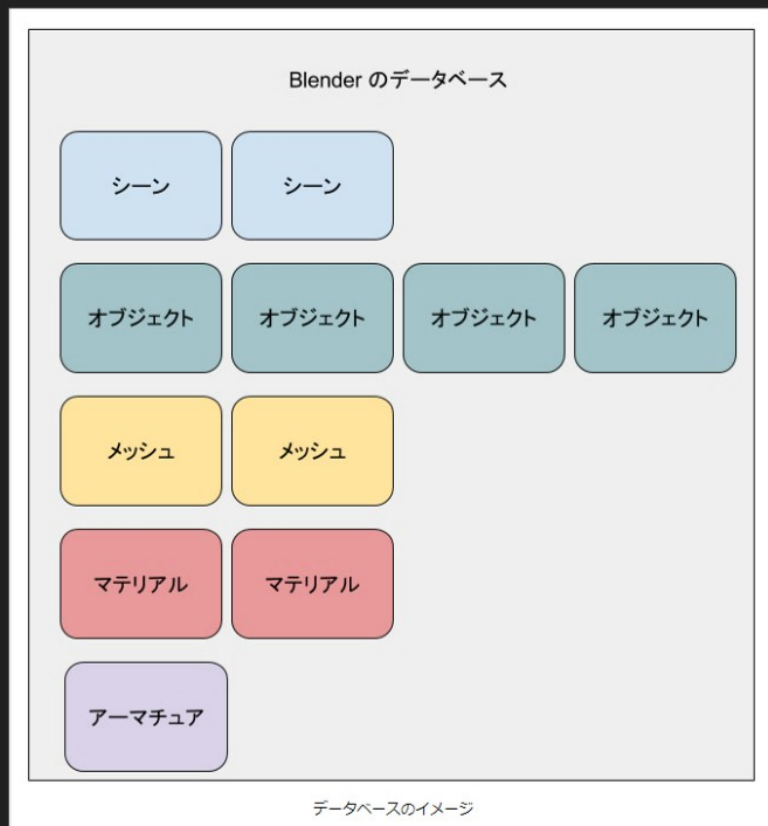
シーンの構成要素をノード（Node:節）とする木構造をシーングラフという。

Unityなどのゲームエンジンや MayaなどのDCCツールには ほぼ確実にシーングラフの構造が 実装されている。

Blenderのノードには、 シーンを頂点として オブジェクト、メッシュ、マテリアル、アーマチュア（スキン）などがある。

データベース

日本工学院専門学校
デザインカレッジ



Blender のあらゆる情報は
データベースに格納されている。

python からは bpy.data を使って
これらのデータベースにアクセスできる。

例えば

オブジェクト： `bpy.data.objects`

レンダリング設定： `bpy.data.scenes`

メッシュ： `bpy.data.meshes`

でアクセスできる。

[リファレンス](#)

予備知識

今回の目標

日本工学院専門学校
デザインカレッジ

- レベルエディタの役割を知る
- Blenderのシーン情報を走査し、表示する

今回の目標

日本工学院専門学校
デザインカレッジ

- レベルエディタの役割を知る
- Blenderのシーン情報を走査し、表示する

docs.google.com – 全画面表示を終了するには、Esc を押します

TL1

01_06.レベルエディタ制作開始

日本工学院専門学校
デザインカレッジ

TL1

01_06.レベルエディタ制作開始

日本工学院専門学校
デザインカレッジ

考えられる改良点

日本工学院専門学校
デザインカレッジ

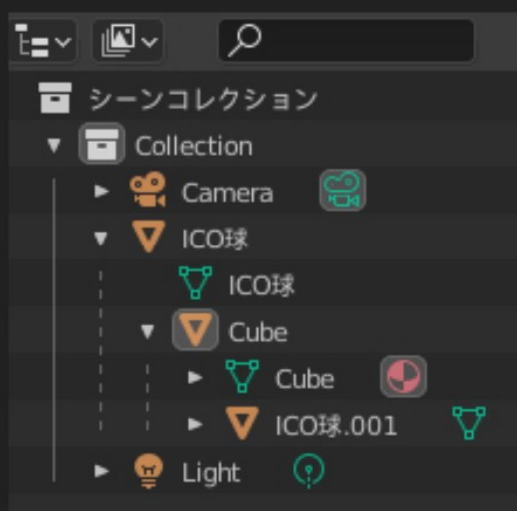
親オブジェクトの名前を表示するようにしたが、
ツールとしての実用性を考えると名前だけで親を検索するのは危険。

Objectの名前はBlender上に自由につけられるものなので、
重複の可能性があるし、ツールのユーザに対して
「重複しないように名前をつけてね」というルールを課すのは無理がある。

全オブジェクトに自動的に通し番号をふって、
親の番号を子供側にデータとして持たせるとよいだろう。

実行結果

日本工学院専門学校
デザインカレッジ



```
シーン情報をExportします
MESH - Cube
Trans(0.000000,0.000000,0.000000)
Rot(0.000000,-0.000000,0.000000)
Scale(1.000000,1.000000,1.000000)
Parent:ICO球

LIGHT - Light
Trans(4.076245,1.005454,5.903862)
Rot(37.261044,3.163707,106.936310)
Scale(1.000000,1.000000,1.000000)

CAMERA - Camera
Trans(7.358891,-6.925791,4.958309)
Rot(63.559303,-0.000003,46.691948)
Scale(1.000000,1.000000,1.000000)

MESH - ICO球
Trans(0.000000,0.000000,0.000000)
Rot(0.000000,-0.000000,0.000000)
Scale(1.000000,1.000000,1.000000)

MESH - ICO球.001
Trans(0.000000,0.000000,0.000000)
Rot(0.000000,-0.000000,0.000000)
Scale(1.000000,1.000000,1.000000)
Parent:Cube

シーン情報をExportしました
```

オブジェクトが親子構造を持つ場合。

「Cube」の親が「ICO球」

「ICO球.001」の親が「Cube」

なので、Blenderでの表示と一致する。

ツリー表示したい場合は、表示関数を再帰関数として実装すればよい。

トランスフォーム情報を表示

日本工学院専門学校
デザインカレッジ

```
+ #トランスフォーム情報を表示
print("Trans(%f,%f,%f)" % (trans.x, trans.y, trans.z) )
print("Rot(%f,%f,%f)" % (rot.x, rot.y, rot.z) )
print("Scale(%f,%f,%f)" % (scale.x, scale.y, scale.z) )
#親オブジェクトの名前を表示
if object.parent:
    print("Parent:" + object.parent.name)
print()
```

前ページの処理で揃えた情報を、差し当たりは `print()` で表示する。

C、C++の `printf` でもおなじみの書式指定子を用いて表示を整える。

また、親オブジェクトがある場合は `parent` 変数にObject型の参照が入っているので、それを使って親オブジェクトの名前も表示してみる。

最後の `print()` は次のオブジェクトとの区切りを入れる為の空行。